

Build Task — HTTP Client Wrapper — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (how a real HTTP client wrapper is actually written, including what makes it satisfy this Build Task's testability requirement). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building one function that other code will call instead of calling `requests` directly: something like `request_with_retry(method, url, ...)`. Its whole job is to hide all the messy failure-handling from this document's exercises behind one clean call.

Inside, it needs to: make the call with a timeout, decide whether a failure is worth retrying, wait longer between each retry, give up after a limit, and turn the final result into either a parsed dictionary or a clear, typed error.

Things to use:

- `requests.request(method, url, timeout=..., **kwargs)` — one function that handles GET/POST/etc. based on the `method` string.
- `response.status_code // 100` — dividing by 100 gives you the status category (2, 4, or 5) without writing out every code.
- `random.uniform(0, 1)` — adds jitter (a small random extra wait) on top of your backoff delay.
- The logger from `01_python_foundations`'s `logging_setup.py` — `get_logger(__name__)`, then `.debug(...)` per attempt, `.error(...)` on final failure.
- Your two new error classes go in `exceptions.py`, just like Doc01's `MissingConfigError`.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're combining everything from this document's exercises into one function:

```
request_with_retry(method: str, url: str, **kwargs) -> dict .
```

The core design decision is a **classification step**: every failure needs to be sorted into "transient" (worth retrying: timeout, 5xx, 429) or "permanent" (not worth retrying: any other 4xx). This classification should probably be its own small piece of logic, separate from the retry loop itself, so you can test it on its own.

The retry loop then wraps a single attempt, using your `compute_backoff_delay(attempt)` helper from the timeout/retry exercise, and stops either on success or once a permanent failure is classified (fail immediately, no retry) or the attempt limit is hit (raise `TransientHTTPError`).

Here's what to actually go look at:

- `requests.request(method, url, timeout=..., **kwargs)` — the generic entry point `requests.get` / `.post` /etc. all call underneath; taking `method` as a string parameter lets your one function handle every HTTP verb.
- **Classifying by status code** — `response.status_code == 429` is transient. `response.status_code // 100 == 5` is transient. `response.status_code // 100 == 4` (and not 429) is permanent. Anything else (2xx, 3xx) is success.
- `requests.exceptions.Timeout` and `requests.exceptions.ConnectionError` — both count as transient failures, same bucket as a 5xx.
- `compute_backoff_delay(attempt) + random.uniform(0, jitter_max)` — jitter is just adding a small random number on top of your existing exponential formula.
- **Attaching the response body to `PermanentHTTPError`** — store it as an attribute (`self.body = response.text`) in `__init__`, so a caller catching the error can actually see what the server said was wrong.

Sketch the function's overall shape — the loop, the classification call, and the two error types it can raise — before checking Hint 2.

Think past "classify the failure and retry it correctly." Ask: **the retry loop reads `Retry-After` if this document's rate-limit exercise applies here, and this function is supposed to be "testable without making real network calls" per the Build Task's own constraint — how do you actually satisfy that, not just claim it?**

The honest answer is a design change, not a testing trick: `request_with_retry()` shouldn't call `requests.request` directly by name — it should accept the transport function as a parameter, defaulting to the real one. A test can then pass in a fake function that returns canned responses

instead of touching the network at all, with zero mocking libraries and zero real HTTP calls. This is the same idea as `compute_backoff_delay(attempt)` being pulled out on its own in the timeout/retry exercise, applied to the *entire network call* this time, not just the backoff math.

There's a second question worth asking too: should every call to `request_with_retry()` open a brand-new connection, or should repeated calls to the same host reuse one, the way the session-reuse exercise showed? A module-level `requests.Session()`, reused across calls, is both faster (connection pooling) and the natural place to set default headers once (like a User-Agent identifying your client) instead of on every call.

The extra pieces:

- A `request_fn` parameter, defaulting to `requests.request`, that the retry loop calls instead of `requests.request` directly — `def request_with_retry(method, url, *, request_fn=requests.request, **kwargs):`
- A module-level `requests.Session()` in `http_client.py`, and `session.request(...)` used as the real `request_fn` instead of the bare module-level function.
- `decide_wait_seconds(response, attempt)` from the rate-limit exercise, called inside the retry loop before falling back to `compute_backoff_delay(attempt)` — a 429 should respect `Retry-After` if the server sent one, the same as any other real client.

Difference between Basic and Intermediate: Basic names the function and the tools. Intermediate designs the classification step and the loop's shape around it, then asks how you'd actually satisfy the Build Task's own "testable without real network calls" constraint (dependency-inject the transport function, rather than hard-coding `requests.request`) and folds in the rate-limit exercise's `Retry-After` handling and the session-reuse exercise's connection pooling — because a real `http_client.py` isn't graded on one exercise's requirements in isolation, it's the one file every later document actually imports.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
exceptions.py:
    make TransientHTTPError, a kind of Exception
    make PermanentHTTPError, a kind of Exception, stores the response body

http_client.py:
    function compute_backoff_delay(attempt):
        return 2 to the power of attempt, plus a small random amount

    function request_with_retry(method, url, max_attempts=5, **kwargs):
        for attempt from 0 to max_attempts - 1:
            log "attempt N" at DEBUG level
            try:
                call requests.request with a timeout
            except timeout or connection error:
                if this was the last attempt: log ERROR, raise TransientHTTPError
                else: wait, try again

            if status is 2xx: return the parsed JSON
            if status is 429 or 5xx:
                if this was the last attempt: log ERROR, raise TransientHTTPError
                else: wait, try again
            if status is any other 4xx:
                log ERROR, raise PermanentHTTPError right away, no retry
```

The trickiest part — turning a bad JSON body into a clear error instead of a raw crash:

```
import json

def parse_json_or_raise(response):
    try:
        return response.json()
    except json.JSONDecodeError as e:
        raise TransientHTTPError("response was not valid JSON: " + str(e))
```

Call this instead of `response.json()` directly inside `request_with_retry`. Try finishing the rest yourself before checking the Intermediate Version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with the classification split into its own function

exceptions.py :

```
class TransientHTTPError is an Exception
class PermanentHTTPError is an Exception, holds status_code and body
```

http_client.py :

```
function compute_backoff_delay(attempt) -> float:
    base = 2 ** attempt
    jitter = a random float between 0 and 1
    return base + jitter

function is_transient_status(status_code) -> bool:
    return status_code == 429 or status_code // 100 == 5

function request_with_retry(method, url, max_attempts=5, **kwargs) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(method, url, timeout=(3, 10), **kwargs)
        except (Timeout, ConnectionError) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            sleep(compute_backoff_delay(attempt))
            continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        if attempt == max_attempts - 1:
            logger.error(
                f"gave up after {max_attempts} attempts, "
```

```

        f"last status {response.status_code}"
    )
    raise TransientHTTPError(
        f"status {response.status_code} after {max_attempts} attempts"
    )
    sleep(compute_backoff_delay(attempt))
    continue

# any other 4xx: permanent, no retry
logger.error(f"permanent failure: status {response.status_code}")
raise PermanentHTTPError(response.status_code, response.text)

```

The key structural idea: the loop has exactly two ways out — a successful return, or a raised error — and exactly one way to keep going — `continue` after sleeping. Nothing falls through silently. Write out the full, typed version yourself, then go one step further — Approach 2, which is what actually satisfies the Build Task's "testable without real network calls" constraint.

Approach 2 — a dependency-injected transport, a shared session, and `Retry-After` support

```

http_client.py:
    module-level: session = requests.Session()  # reused across every call

    function request_with_retry(
        method, url, max_attempts=5, request_fn=session.request, **kwargs
    ) -> dict:
        for attempt in range(max_attempts):
            logger.debug(f"attempt {attempt}: {method} {url}")
            try:
                response = request_fn(method, url, timeout=(3, 10), **kwargs)
            except (Timeout, ConnectionError) as e:
                if attempt == max_attempts - 1:
                    raise TransientHTTPError(...) from e
                sleep(compute_backoff_delay(attempt))
                continue

            if response.status_code < 300:
                return parse_json_or_raise(response)

            if is_transient_status(response.status_code):
                if attempt == max_attempts - 1:
                    raise TransientHTTPError(...)
                # respects Retry-After if the server sent one
                wait = decide_wait_seconds(response, attempt)
                sleep(wait)
                continue

```

```
raise PermanentHTTPError(response.status_code, response.text)
```

Here's almost the whole thing — fill in the missing piece yourself:

```
import json
import random
import time
from typing import Any, Callable

import requests

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)
_session = requests.Session()

def compute_backoff_delay(attempt: int) -> float:
    return (2 ** attempt) + random.uniform(0, 1)

def is_transient_status(status_code: int) -> bool:
    return status_code == 429 or status_code // 100 == 5

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    request_fn: Callable[..., requests.Response] = _session.request,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = request_fn(method, url, timeout=(3, 10), **kwargs)
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
```

```

        continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        # your turn: on the last attempt, log ERROR and raise
        # TransientHTTPError; otherwise sleep and continue – for the
        # sleep, reuse decide_wait_seconds(response, attempt) from the
        # rate-limit exercise so a real Retry-After header is respected
        ...

    logger.error(f"permanent failure: status {response.status_code}")
    raise PermanentHTTPError(response.status_code, response.text)

```

Fill in the transient branch yourself (a fake `request_fn` that returns canned `Response`-like objects is exactly how you'd unit test this without a real network call), then compare all of your finished versions against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode calls `requests.request` by name and never respects `Retry-After`. Intermediate Approach 1 is the same shape, fully typed, with the transient/permanent split as its own function. Approach 2 injects the transport call as a parameter (`request_fn`) so the whole function becomes testable with a fake in place of the network, reuses a module-level `Session` for connection pooling, and folds in `Retry-After` handling from the rate-limit exercise — the combination this document's 5 exercises were always building toward.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows what you'd see running it against a real, working call unless a block says otherwise. Read both depths — they're not "wrong, right," they're 2 real, valid stages of building the same client, with real tradeoffs between them.

Basic Version

Approach 1 — one function, everything inline

Story — `exceptions.py` : `request_with_retry()` needs a way to tell its caller "this is worth retrying elsewhere too" apart from "this will never work, stop trying" — two error types make that distinction part of the function's actual contract, not just a comment. **If not:** every later document that imports `http_client.py` would catch one generic exception and have no way to tell a transient network hiccup from its own bad request.

```
# exceptions.py
# Why: two separate types so callers can catch "worth retrying" and
# "my mistake, don't retry" differently, instead of one generic Exception.
class TransientHTTPError(Exception):
    pass

class PermanentHTTPError(Exception):
    def __init__(self, status_code, body):
        self.status_code = status_code
        self.body = body
        super().__init__("HTTP " + str(status_code) + ": " + body)
```

Story — `http_client.py` : this is the Goal of the whole Build Task — one wrapper around `requests` that every later document imports instead of calling the network directly, so timeout/retry/classification logic exists exactly once in the whole curriculum. **If not:** every later document (Doc04's `chat_client.py`, and beyond) would re-implement its own retry loop, each slightly different, each with its own undiscovered bugs.

```
# http_client.py
import json
import random
import time
import requests
```

```

from exceptions import TransientHTTPError, PermanentHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

def compute_backoff_delay(attempt):
    # Why: spaces retries out (2, 4, 8... seconds) with a little randomness,
    # so a burst of clients retrying together doesn't all hit the server at
    # once.
    return (2 ** attempt) + random.uniform(0, 1)

def request_with_retry(method, url, max_attempts=5, **kwargs):
    # Why: the one function every later document calls instead of requests
    # directly – hides the timeout/retry/classify logic behind one clean call.
    for attempt in range(max_attempts):
        logger.debug("attempt " + str(attempt) + ": " + method + " " + url)
        try:
            response = requests.request(method, url, timeout=(3, 10), **kwargs)
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(
                    "gave up after " + str(max_attempts) + " attempts: " + str(e)
                )
                raise TransientHTTPError(str(e))
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError("bad JSON in response: " + str(e))

        if response.status_code == 429 or response.status_code // 100 == 5:
            if attempt == max_attempts - 1:
                logger.error("gave up, last status " + str(response.status_code))
                raise TransientHTTPError("status " + str(response.status_code))
            time.sleep(compute_backoff_delay(attempt))
            continue

        logger.error("permanent failure: status " + str(response.status_code))
        raise PermanentHTTPError(response.status_code, response.text)

```

```
if __name__ == "__main__":  
    print(request_with_retry("GET", "https://api.github.com"))
```

Expected output:

```
{'current_user_url': 'https://api.github.com/user', ...}
```

This version works correctly and meets every Build Task requirement. It's missing type hints, doesn't separate the transient/permanent decision into its own testable function, and calls `requests.request` directly, which makes it awkward to test without a real network call — all fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — everything typed, still inline

exceptions.py

```
class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):
    """Raised immediately for a non-retryable 4xx, with the response body
    attached."""

    def __init__(self, status_code: int, body: str) -> None:
        self.status_code = status_code
        self.body = body
        super().__init__(f"HTTP {status_code}: {body[:200]}")
```

http_client.py

```
import json
import random
import time
from typing import Any

import requests

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

DEFAULT_TIMEOUT = (3, 10)

def compute_backoff_delay(attempt: int) -> float:
    return (2 ** attempt) + random.uniform(0, 1)
```

```

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if response.status_code == 429 or response.status_code // 100 == 5:
            if attempt == max_attempts - 1:
                logger.error(f"gave up, last status {response.status_code}")
                raise TransientHTTPError(
                    f"status {response.status_code} after {max_attempts} attempts"
                )
            time.sleep(compute_backoff_delay(attempt))
            continue

        logger.error(f"permanent failure: status {response.status_code}")
        raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))

```

Why this approach: everything lives in one place, which is easy to read start to finish for a project this size.

Approach 2 — the transient/permanent decision pulled into its own testable function

```

def is_transient_status(status_code: int) -> bool:
    """A 429, or any 5xx, is worth retrying. Everything else that isn't a
    success is a permanent failure the caller needs to fix, not retry."""
    return status_code == 429 or status_code // 100 == 5

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if is_transient_status(response.status_code):
            if attempt == max_attempts - 1:
                logger.error(f"gave up, last status {response.status_code}")
                raise TransientHTTPError(
                    f"status {response.status_code} after {max_attempts} attempts"
                )
            time.sleep(compute_backoff_delay(attempt))
            continue

```

```
logger.error(f"permanent failure: status {response.status_code}")
raise PermanentHTTPError(response.status_code, response.text)
```

Why this approach: `is_transient_status(429)` and `is_transient_status(404)` can be tested directly with plain integers — no fake `Response` object, no mocking `requests` at all. This is real progress toward the Build Task's "testable without real network calls" constraint, but it's still not the whole answer — `request_with_retry` itself still calls `requests.request` by name, so testing the *loop* (does it really stop after `max_attempts` ? does it really sleep between tries?) still means either mocking `requests.request` globally or making real calls.

Difference from Basic: both Intermediate approaches add full type hints and a `DEFAULT_TIMEOUT` constant instead of a repeated magic tuple. Approach 2 additionally separates the transient/permanent decision into its own pure function, testable in complete isolation from the network.

Approach 3 — a dependency-injected transport, a shared session, and `Retry-After` support

```
# exceptions.py
# Why: two separate types so callers can catch "worth retrying" and
# "my mistake, don't retry" differently, instead of one generic Exception.
class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):
    """Raised immediately for a non-retryable 4xx, with the response body
    attached."""

    def __init__(self, status_code: int, body: str) -> None:
        self.status_code = status_code
        self.body = body
        super().__init__(f"HTTP {status_code}: {body[:200]}")
```

```
# http_client.py
import json
import random
import time
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime
from typing import Any, Callable

import requests
```

```

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

DEFAULT_TIMEOUT = (3, 10)
MAX_RETRY_AFTER_SECONDS = 60.0

_session = requests.Session()
_session.headers.update({"User-Agent": "learning-langgraph-http-client/1.0"})

def compute_backoff_delay(attempt: int) -> float:
    # Why: spaces retries out (2, 4, 8... seconds) with a little randomness,
    # so a burst of clients retrying together doesn't all hit the server at
    # once.
    return (2 ** attempt) + random.uniform(0, 1)

def is_transient_status(status_code: int) -> bool:
    # Why: pulled out on its own so it's testable with plain integers –
    # no fake Response object, no network, no mocking needed.
    return status_code == 429 or status_code // 100 == 5

def _parse_retry_after(value: str) -> float | None:
    # Why: the HTTP spec allows Retry-After to be a plain number of seconds
    # OR an HTTP-date string – this handles both instead of crashing on one.
    try:
        return float(value)
    except ValueError:
        pass
    try:
        target_time = parsedate_to_datetime(value)
        seconds_left = (target_time - datetime.now(timezone.utc)).total_seconds()
        return max(seconds_left, 0.0)
    except (TypeError, ValueError):
        return None

def decide_wait_seconds(response: requests.Response, attempt: int) -> float:
    # Why: respects a real Retry-After header when the server sends one,
    # and falls back to exponential backoff (capped) when it doesn't.
    retry_after = response.headers.get("Retry-After")
    wait = _parse_retry_after(retry_after) if retry_after is not None else None
    if wait is None:
        wait = compute_backoff_delay(attempt)

```

```

return min(wait, MAX_RETRY_AFTER_SECONDS)

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    request_fn: Callable[..., requests.Response] = _session.request,
    **kwargs: Any,
) -> dict:
    # why: the one function every later document imports instead of calling
    # requests directly – request_fn is injectable so tests never touch the
    # network.
    for attempt in range(max_attempts):
        # how: DEBUG, not print – this is diagnostic, not output
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            # how: request_fn defaults to the shared _session.request, but a
            # test can pass a fake function here instead – no real network call.
            response = request_fn(method, url, timeout=DEFAULT_TIMEOUT, **kwargs)
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            # when: the request itself never got a response at all –
            # treated as transient, same bucket as a 5xx status.
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            # how: goes back to the top of the loop for the next attempt
            continue

    if response.status_code < 300:
        # when: 2xx/3xx is success – try to parse it and return.
        try:
            return response.json()
        except json.JSONDecodeError as e:
            # why: a 200 with an unparseable body is still a real
            # failure the caller needs to know about, not a silent None.
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        # when: 429 or 5xx – worth retrying, unless this was the last
        # attempt.
        if attempt == max_attempts - 1:
            logger.error(f"gave up, last status {response.status_code}")

```

```

        raise TransientHTTPError(
            f"status {response.status_code} after {max_attempts} attempts"
        )
        # how: honors Retry-After if the server sent one
        time.sleep(decide_wait_seconds(response, attempt))
        continue

    # when: any other 4xx – this is a client mistake that will fail
    # again the same way, so raise immediately instead of retrying.
    logger.error(f"permanent failure: status {response.status_code}")
    raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))

```

Expected output:

```
{'current_user_url': 'https://api.github.com/user', ...}
```

Story — `test_http_client.py`: `http_client.py` is about to get imported by every later document — Doc04's `chat_client.py` calls through it for every OpenAI request. A regression here (the loop stops retrying too early, a 404 gets retried when it shouldn't) needs to be caught right here, in milliseconds, not discovered three documents later against a real API. **If not:** the only way to know the retry logic still works would be to run it against a real, possibly flaky network — slow, unreliable, and not something you'd run on every change.

`test_http_client.py` , proving the network never has to be touched:

```

import pytest

from exceptions import PermanentHTTPError, TransientHTTPError
from http_client import is_transient_status, request_with_retry

class FakeResponse:
    def __init__(self, status_code, json_body=None, text="", headers=None):
        self.status_code = status_code
        self._json_body = json_body
        self.text = text
        self.headers = headers or {}

    def json(self):
        return self._json_body

```

```

def test_is_transient_status():
    assert is_transient_status(429) is True
    assert is_transient_status(500) is True
    assert is_transient_status(503) is True
    assert is_transient_status(404) is False
    assert is_transient_status(200) is False

def test_succeeds_immediately():
    def fake_request(method, url, timeout, **kwargs):
        return FakeResponse(200, json_body={"ok": True})

    result = request_with_retry(
        "GET", "https://example.invalid", request_fn=fake_request
    )
    assert result == {"ok": True}

def test_retries_429_then_succeeds():
    calls = {"count": 0}

    def fake_request(method, url, timeout, **kwargs):
        calls["count"] += 1
        if calls["count"] == 1:
            return FakeResponse(429, headers={"Retry-After": "0"})
        return FakeResponse(200, json_body={"ok": True})

    result = request_with_retry(
        "GET", "https://example.invalid", request_fn=fake_request
    )
    assert result == {"ok": True}
    assert calls["count"] == 2

def test_404_fails_immediately_no_retry():
    calls = {"count": 0}

    def fake_request(method, url, timeout, **kwargs):
        calls["count"] += 1
        return FakeResponse(404, text="not found")

    with pytest.raises(PermanentHTTPError):
        request_with_retry(
            "GET", "https://example.invalid", request_fn=fake_request
        )
    assert calls["count"] == 1

```

```
def test_gives_up_after_max_attempts():
    def fake_request(method, url, timeout, **kwargs):
        return FakeResponse(500, text="server error")

    with pytest.raises(TransientHTTPError):
        request_with_retry(
            "GET",
            "https://example.invalid",
            max_attempts=3,
            request_fn=fake_request,
        )
```

Expected output (`pytest test_http_client.py -v`): all 5 tests pass, in well under a second — not one of them opens a real socket, because `request_fn` is a plain Python function each test controls completely.

Difference from Approach 2: Approach 2's `request_with_retry` calls `requests.request` by name, so testing the *loop itself* — does it really retry a 429, does it really give up after `max_attempts`, does it really leave a 404 alone — means either making real HTTP calls or reaching for a mocking library. Approach 3 accepts the transport call as a parameter (`request_fn`, defaulting to a shared `Session`'s `.request`), so `test_http_client.py` above tests the actual retry logic with zero network calls and zero mocking libraries — genuinely satisfying the Build Task's constraint instead of only satisfying part of it. It also folds in `decide_wait_seconds()` from the rate-limit exercise, so a real 429 with a `Retry-After` header is honored (and capped) instead of always falling back to blind exponential backoff, and reuses one `Session` across every call for connection pooling and a consistent `User-Agent`.

Which one should you actually write? Approach 2 is a completely reasonable place to stop if this file will only ever be tested by hand, against the real network, during development. But this file is imported by every later document in this curriculum — Doc04's `chat_client.py` calls through it for every OpenAI request — so real, fast, reliable tests matter here more than almost anywhere else in the project. Approach 3's `request_fn` injection costs one extra parameter and buys you a test suite that runs in milliseconds and never depends on a real server being up, which is why it's the version that actually satisfies this Build Task's stated requirements. Write it this way.